

Random extras

COMP 4630 | Winter 2026

Charlotte Curtis

Overview

- Object detection
- Autoencoders
- Generative adversarial networks (GANs)
- Diffusion models
- References and suggested reading:
 - [Scikit-learn book](#): Chapter 17
 - [Deep Learning book](#): Chapter 14, 20

But first, some housekeeping

- Assignment 3 stuff

```
class SomeModel(nn.Module):  
    def __init__(self):  
        super(SomeModel, self).__init__()  
  
        self.bert = transformers.BertModel.from_pretrained("bert-base-uncased")  
        self.out = nn.Linear(self.bert.config.hidden_size, 1)
```

- Final checkpoint and report
- Journal club participation grade

Object Detection

R(egion)-CNN

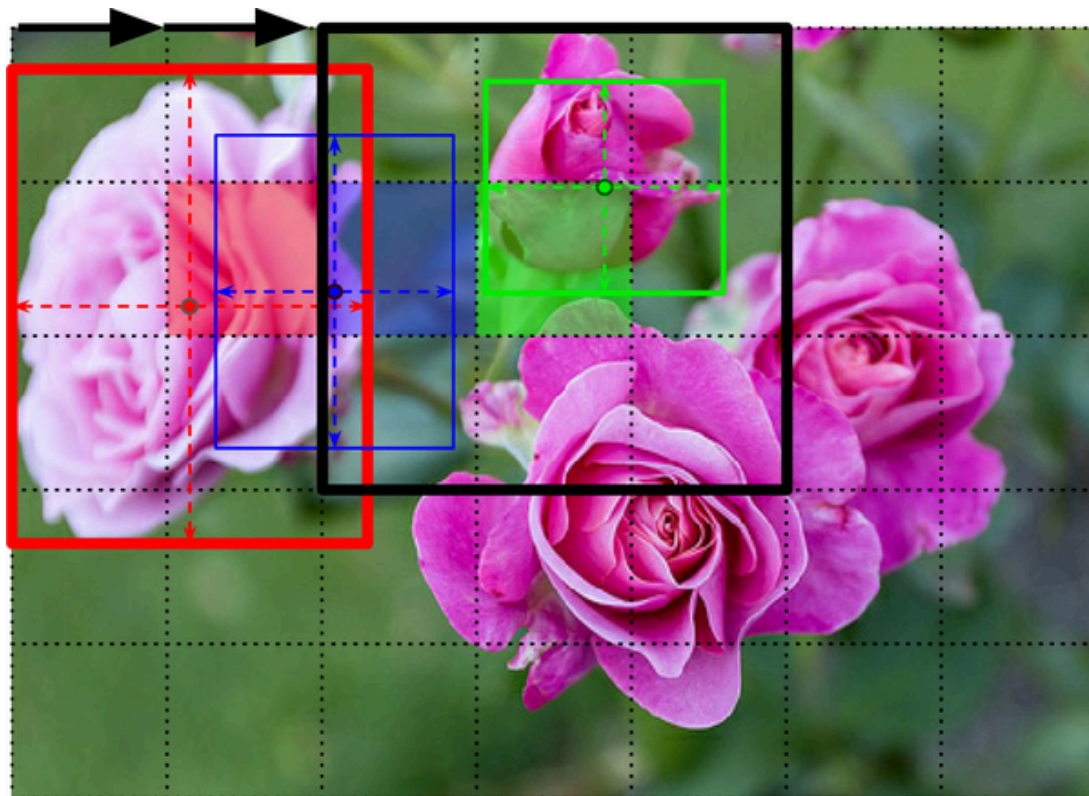


Fig 12-25 from Scikit-learn with Pytorch

You Only Look Once (YOLO)

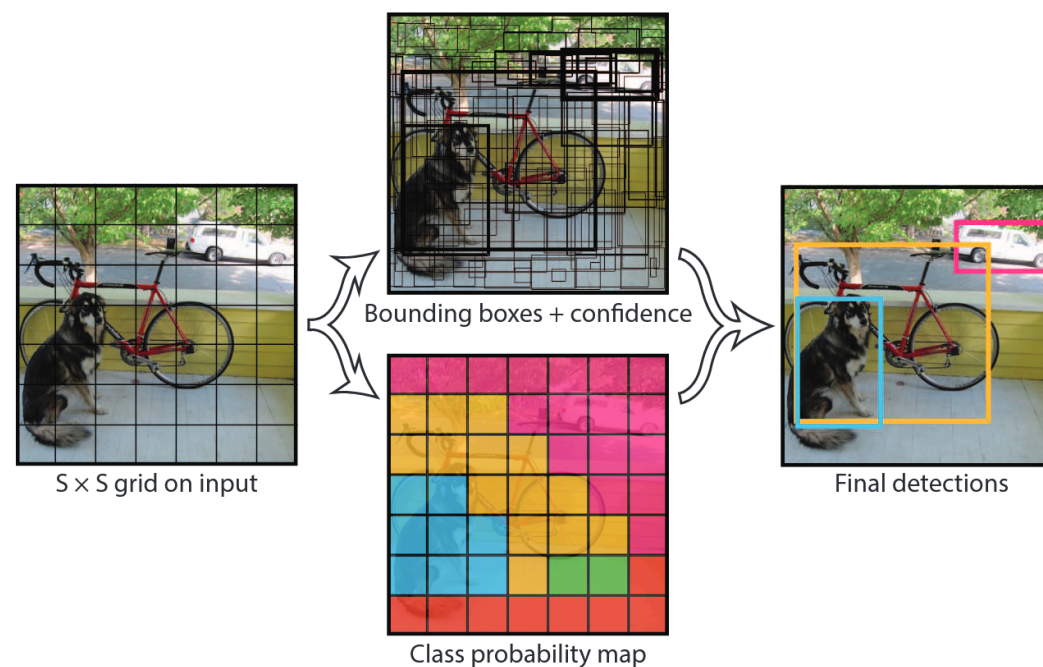


Fig 1 from [Redmon et al](#)

Autoencoders

- Like encoder-decoder networks, but the input and output are the same, minimizing:

$$\mathcal{L}(\mathbf{x}, f(g(\mathbf{x})))$$

where f is the encoder and g is the decoder

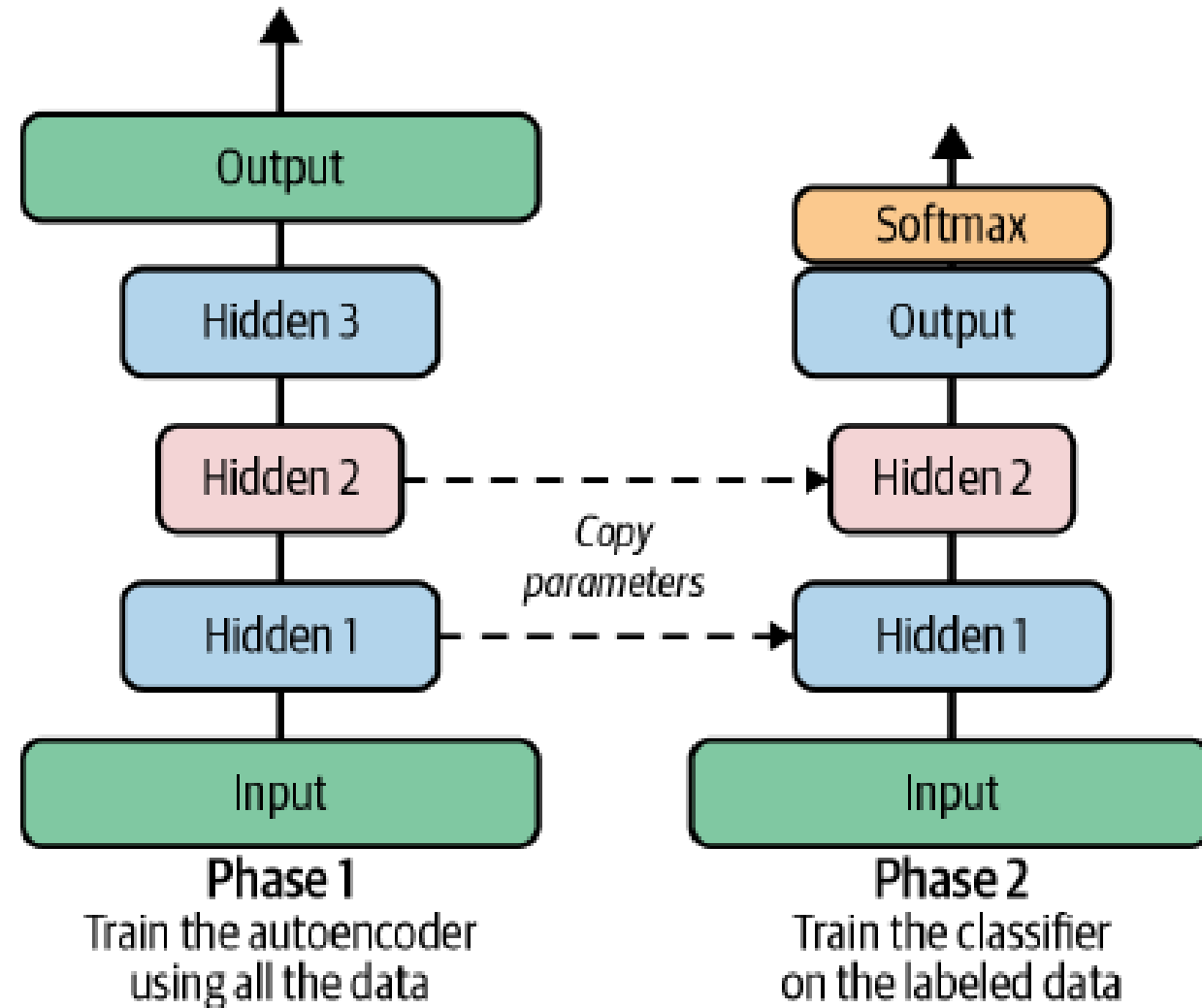
- Autoencoders have been around [since the 80s](#)
- **?** If we're just training a model to copy its input, what's the point?

What can we do with autoencoders?

- Autoencoders learn a **latent space** representation of the data, provided the latent space is **lower dimension** than the input (undercomplete)
- **Regularization** can also help the model learn a useful representation
- Once trained, the model can be used for:
 - Data compression
 - Information retrieval
 - Anomaly detection
 - Unsupervised pretraining

Unsupervised pretraining

- Rather than training (e.g.) a classifier from scratch, we can first train an **autoencoder**, then use the **encoder** part as the first half of the classifier



Variational Autoencoders

- Encoder produces a **distribution** (μ, σ)
- Decoder samples from $N(\mu, \sigma)$ to produce the output
- Loss includes a **KL divergence** term to push the distribution towards normal

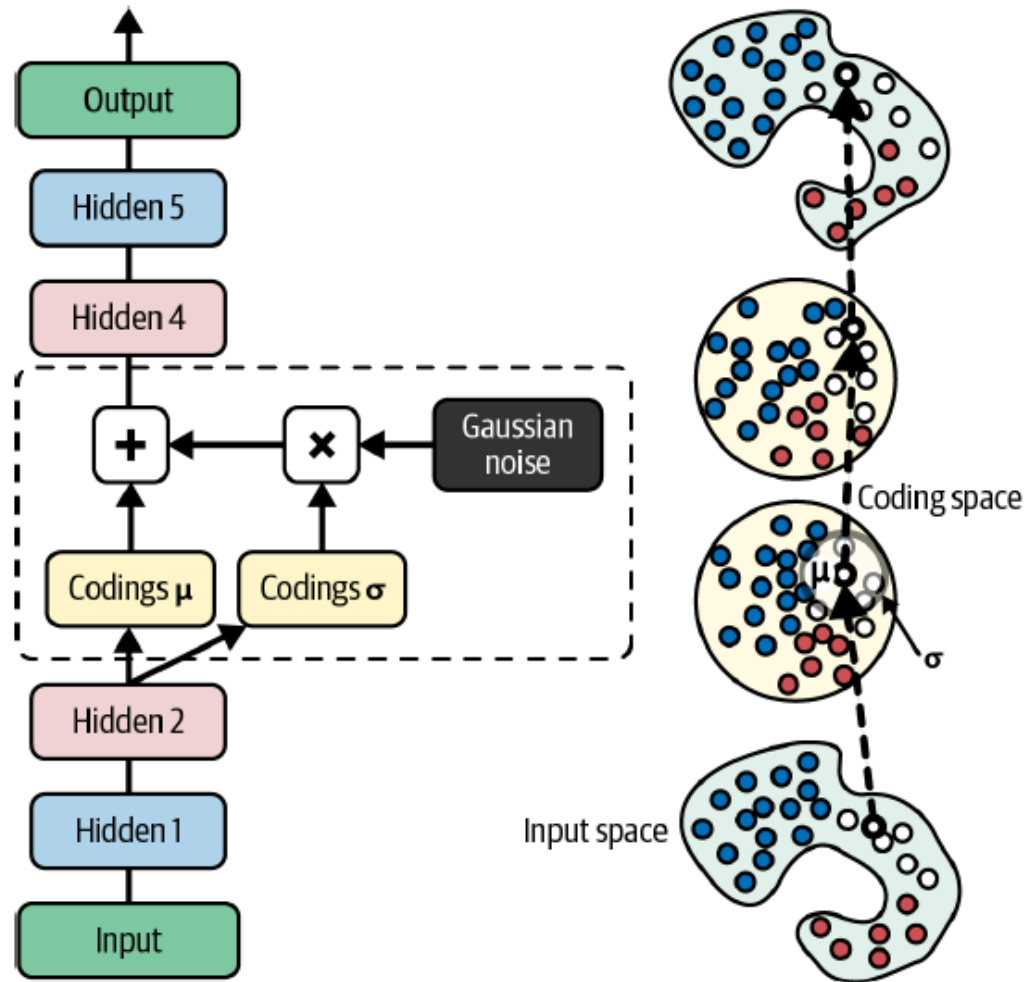


Figure 17-11. A variational autoencoder (left) and an instance going through it (right)

Generative Adversarial Networks (GANs)

- VAEs can be used as generators (e.g. images), so how can we train them to generate what we actually want?
- In [2014](#), Ian Goodfellow et. al proposed training two networks simultaneously: a **generator** and a **discriminator**
 - The generator tries to produce realistic looking images
 - The discriminator is trained to classify images as real or fake
- Two-phase training loop:
 - Equal numbers of real and fake images used to train discriminator
 - Discriminator weights frozen while generator produces images with label of "real" - backpropagation updates only generator weights

GAN training challenges

- Training is a **zero-sum** game, where each "player" receives a payoff that the discriminator tries to maximize and the generator tries to minimize
- Equilibrium is reached when the discriminator performance is 50%, but **oscillations** can occur and convergence is not guaranteed
- **Mode collapse** happens when the generator figures out that it can produce a single image that fools the discriminator
- Still an active area of research!

"And since many factors affect these complex dynamics, GANs are very sensitive to the hyperparameters: you may have to spend a lot of effort fine-tuning them. In fact, that's why I used RMSProp rather than Nadam when compiling the models: when using Nadam, I ran into a severe mode collapse." - Aurélien Géron

Diffusion Models

- First formalized in 2015 by Jascha Sohl-Dickstein et. al
- Not popular until 2020 when Jonathan Ho et. al introduced **DDPM**
- Concept: take an image and gradually add noise until it's unrecognizable, then train a model to reverse the process

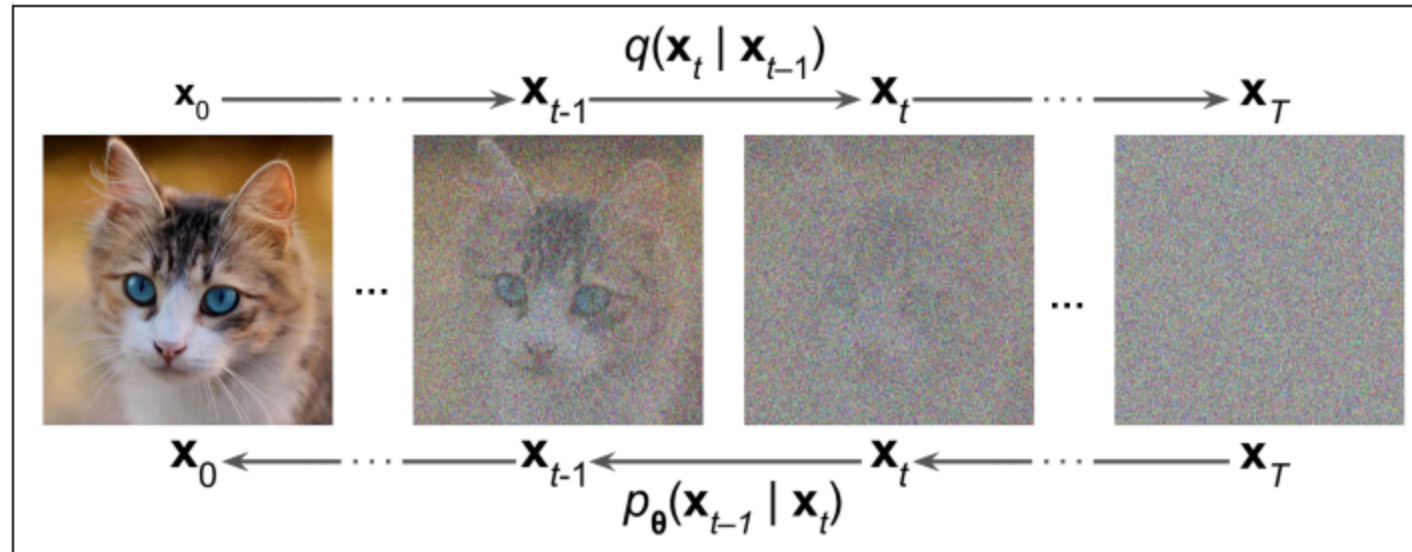


Figure 17-20. The forward process q and reverse process p

CLIP guidance

- DDPM, GANs, and VAEs are all capable of generating images given a **class label** that is part of the training data
- Image generation + NLP = magic
- Contrastive Language-Image Pretraining **CLIP** was trained on 400 million image-caption pairs to predict which caption goes with which image
- Final result is a general purpose model that can go both ways
- Actually **open sourced** by OpenAI!

That's all, next up is your project presentations!
